

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1442
Higher

COURSE NAME: Compiler Design for

Level Processing

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I G RAM KISHORE 192424385(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G RAM KISHORE

Course Faculty

Q1. Predictive Parser Design

```
#include <stdio.h>

char in[] = "i+i*i$"; char st[50]; int top = -1, ip
= 0; void push(char c){ st[++top]=c; } void
pop(){ top--; } int main() {   push('$');
push('E');   while(top!=-1){      char x =
st[top];      if(x==in[ip]) { pop(); ip++; }
else if(x=='E'){ pop(); push('A'); push('T'); }
else if(x=='A'){      pop();
        if(in[ip]=='+'){ push('A'); push('T'); push('+'); }
        }
        else if(x=='T'){ pop(); push('B'); push('F'); }
else if(x=='B'){      pop();
        if(in[ip]=='*'){ push('B'); push('F'); push('*'); }
        }
        else if(x=='F'){
pop();
if(in[ip]=='i') push('i');
else{      push('');
push('E');
push('(');
        }
}
        else{
        printf("String Rejected");
return 0;
        }
}
if(in[ip]=='$')
printf("String Accepted");
```

```
else    printf("String
Rejected");    return 0;
}
```

The screenshot displays the Programiz C Online Compiler interface. The editor contains a C program implementing a stack using an array.

```
main.c  
1 #include <stdio.h>  
2  
3 char in[] = "i-i!i$";  
4 char st[50];  
5 int top = -1, ip = 0;  
6  
7 void push(char c){ st[++top]=c; }  
8 void pop(){ top--; }  
9  
10 int main() {  
11     push('$'); push('E');  
12  
13     while(top!=-1){  
14         char x = st[top];  
15  
16         if(x==in[ip]) { pop(); ip++; }  
17  
18         else if(x=='E'){ pop(); push('A'); push('T'); }  
19         else if(x=='A'){  
20             pop();  
21             if(in[ip]!='') { push('A'); push('T'); push('+'); }  
22         }  
23         else if(x=='T'){ pop(); push('B'); push('F'); }  
24         else if(x=='B'){  
25             pop();  
26             if(in[ip]!='') { push('B'); push('F'); push('*'); }  
27         }  
28         else if(x=='F'){  
29             pop();  
30             if(in[ip]=='i') push('i');  
31             else{  
32                 push('(');  
33                 push('E');
```

The output window shows the following messages:

```
String Rejected  
=== Code Execution Successful ===
```

Q3. LR parsing tables

```
#include <stdio.h>

#include <string.h> int

main() {

    char input[] = "id=*id";

    if(strcmp(input,"id=*id")==0) {

        printf("Stack\t\tInput\t\tAction\n");    printf("-----
        -----\n");    printf("$\t\tid=*id$\t\tShift

id\n");    printf("$id\t\t=*id$\t\tShift =\n");

        printf("$id=\t\t*id$\t\tShift *\n");

        printf("$id=*id$\t\tid$\t\tShift id\n");

        printf("$id=*id$\t\tid$\t\tReduce R->L\n");

        printf("$id=*R$\t\tid$\t\tReduce L->*R\n");

        printf("$id=L$\t\tid$\t\tReduce S->L=R\n");

        printf("$S$\t\tid$\t\tAccept\n");

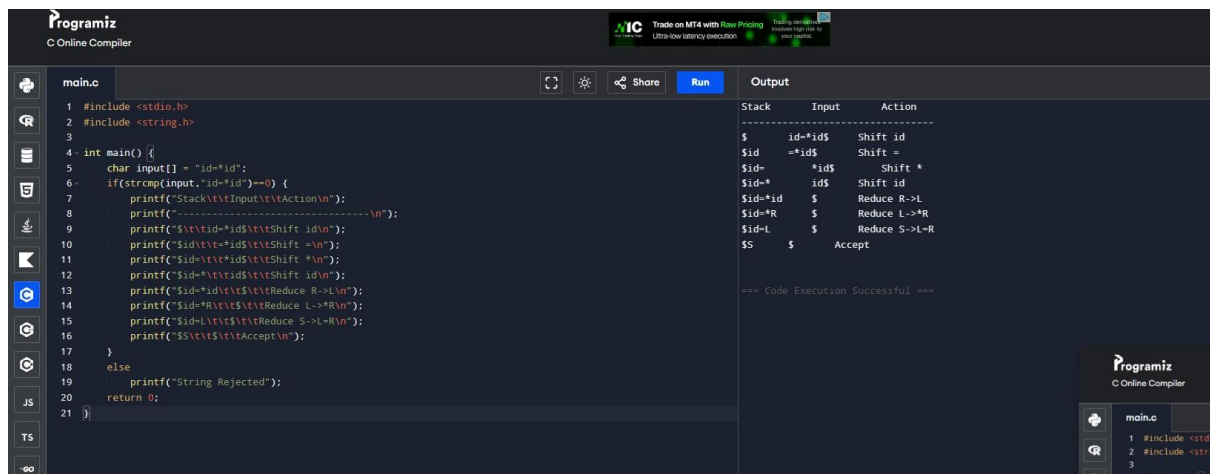
    }

    else

        printf("String Rejected");

    return 0;

}
```



The screenshot shows the Programiz C Online Compiler interface. The code in main.c is as follows:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char input[] = "id=*id";
6     if(strcmp(input,"id=*id")==0) {
7         printf("Stack\t\tInput\t\tAction\n");
8         printf("-----\n");
9         printf("$\t\tid=*id$\t\tShift id\n");
10        printf("$id=\t\t*id$\t\tShift =\n");
11        printf("$id=*id$\t\tid$\t\tShift *\n");
12        printf("$id=L$\t\tid$\t\tShift id\n");
13        printf("$id=*id$\t\tid$\t\tReduce R->L\n");
14        printf("$id=*R$\t\tid$\t\tReduce L->*R\n");
15        printf("$id=L$\t\tid$\t\tReduce S->L=R\n");
16        printf("$S$\t\tid$\t\tAccept\n");
17    }
18    else
19        printf("String Rejected");
20    return 0;
21 }
```

The output on the right shows the execution of the code:

Stack	Input	Action
\$	id=*id\$	Shift id
\$id	=*id\$	Shift =
\$id=	*id\$	Shift *
\$id=*	id\$	Shift id
\$id=*id	\$	Reduce R->L
\$id=*R	\$	Reduce L->*R
\$id=L	\$	Reduce S->L=R
\$S	\$	Accept

==== Code Execution Successful ====

Q8. Ambiguity Removal and Parsing Strategy Evaluation

```
#include <stdio.h>

#include <string.h> int

main()

{

    char input[20];    printf("Enter
Input: ");    scanf("%s", input);
if(strcmp(input, "id-id/id") == 0)
    {
        printf("\nShift-Reduce Parsing\n");
printf("-----\n");
printf("Shift id\n");    printf("Reduce
E -> id\n");    printf("Shift -\n");
printf("Shift id\n");    printf("Reduce
E -> id\n");    printf("Shift /\n");
printf("Shift id\n");    printf("Reduce
E -> id\n");    printf("Reduce T -> T /
F\n");    printf("Reduce E -> E - T\n");
printf("\nString Accepted\n");
    }
    else
    {
        printf("\nString Rejected\n");
    }
    return 0;
}
```

Programiz

C Online Compiler

Fly for a stay. Stay for a flight.

Introducing Two-way point conversion.

Convert now

16C apply

main.c

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char input[20];
7
8     printf("Enter Input: ");
9     scanf("%s", input);
10
11     if(strcmp(input, "id-id/id") == 0)
12     {
13         printf("\nShift-Reduce Parsing\n");
14         printf("-----\n");
15         printf("Shift id\n");
16         printf("Reduce E -> id\n");
17
18         printf("Shift /\n");
19
20         printf("Shift id\n");
21         printf("Reduce E -> id\n");
22
23         printf("Shift /\n");
24
25         printf("Shift id\n");
26         printf("Reduce E -> id\n");
27     }
28 }
```

Output

Enter Input: id-id/id
Shift-Reduce Parsing

Shift id
Reduce E -> id
Shift /
Shift id
Reduce E -> id
Shift /
Shift id
Reduce E -> id
Reduce T -> T / F
Reduce E -> E - T

String Accepted

=== Code Execution Successful ===

CONCEPT MAP

```

graph TD
    CD((COMPILER DESIGN))
    CD --- 1[1 INTRODUCTION  
• Compilers  
• Language Processors  
• Phases of a compiler  
• Grouping of Phases]
    CD --- 2[2 LEXICAL ANALYSIS  
• Compiler construction tools  
• Lexical Analysis  
• Role of Lexical Analyzer  
• Specification of Tokens  
• LEX]
    CD --- 3[3 SYNTAX ANALYSIS  
• Role of the parser  
• Context-Free Grammars  
• Recursive Descent Parsing  
• Predictive Parsing  
• Shift Reduce Parsing  
• LR & SLR Parsers  
• Operator Precedence Parsing & YACC]
    CD --- 4[4 ERROR HANDLING  
• Error Handling and Recovery in Syntax Analyzer]
    CD --- 5[5 SYNTAX DIRECTED TRANSLATION  
• Syntax Directed Definitions  
• Construction of Syntax Trees  
• Bottom-up Evaluation of S-Attributed and L-Attributed Definitions]
    CD --- 6[6 INTERMEDIATE CODE GENERATION  
• Top-down Translation]
    CD --- 7[7 TYPE SYSTEMS  
• Specification of a simple type checker  
• Equivalences of type expressions  
• Type conversions]
    CD --- 8[8 DECLARATIONS]
    CD --- 9[9 ASSIGNMENT STATEMENTS & BOOLEAN EXPRESSIONS  
• Assignment Statements  
• Boolean Expressions]
    CD --- 10[10 CONTROL STATEMENTS  
• Case Statements  
• Back patching  
• Procedure Calls]
    CD --- 11[11 IMPLEMENTATION TOOLS]
    CD --- 12[12 PROGRAMMING ENVIRONMENTS]
    CD --- 13[13 LANGUAGE SPECIFICATIONS]
    CD --- 14[14 INTERMEDIATE LANGUAGES]
    CD --- 15[15 BACK END  
• Code Generation  
• Machine Dependent Optimization]
    CD --- 16[16 MACHINE DEPENDENT OPTIMIZATION]
    CD --- 17[17 OPTIMIZATION  
• Optimization of Basic Blocks  
• Basic Blocks and Flow Graphs]
    CD --- 18[18 OPTIMIZATION TECHNIQUES  
• Local Optimization  
• Intermediate Code Improvement]
    CD --- 19[19 DATA FLOW ANALYSIS  
• Basic Blocks and Flow Graphs  
• Global Data Flow Analysis]
    CD --- 20[20 CODE GENERATION  
• The target machine  
• Runtime Storage management  
• Simple Code Generation Algorithm  
• Next-use Information]
    CD --- 21[21 RUNTIME STORAGE MANAGEMENT  
• Issues in the Design of Code Generator]
    CD --- 22[22 CODE OPTIMIZATION  
• Principal Sources of Optimization  
• Optimization of Basic Blocks]
    CD --- 23[23 BASIC BLOCKS  
• DAG representation of Basic Blocks]
  
```

CONCEPT MAP

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation